HealthRAG: Knowledge-Driven Medical QA System

Members and Instructor



PGS. TS. Phan Duy Hùng



Nguyễn Mạnh Tuấn
HE171284



Lê Việt Hưng
HE170038



Trần Bá Cường
HE171454



Trương Hữu Nhật
HE170351

OverView

1. Introduction

- Problem & Motivation
- Research Objectives

2. Theoretical Foundation

- RAG Overview
- Retrieval Methods

3. Methodology

- System Architecture
- Data Collection & Preprocessing
- Model Fine-tuning

4. Results & Analysis

- Metric Results and Analysis

5. Demo App

Introduction

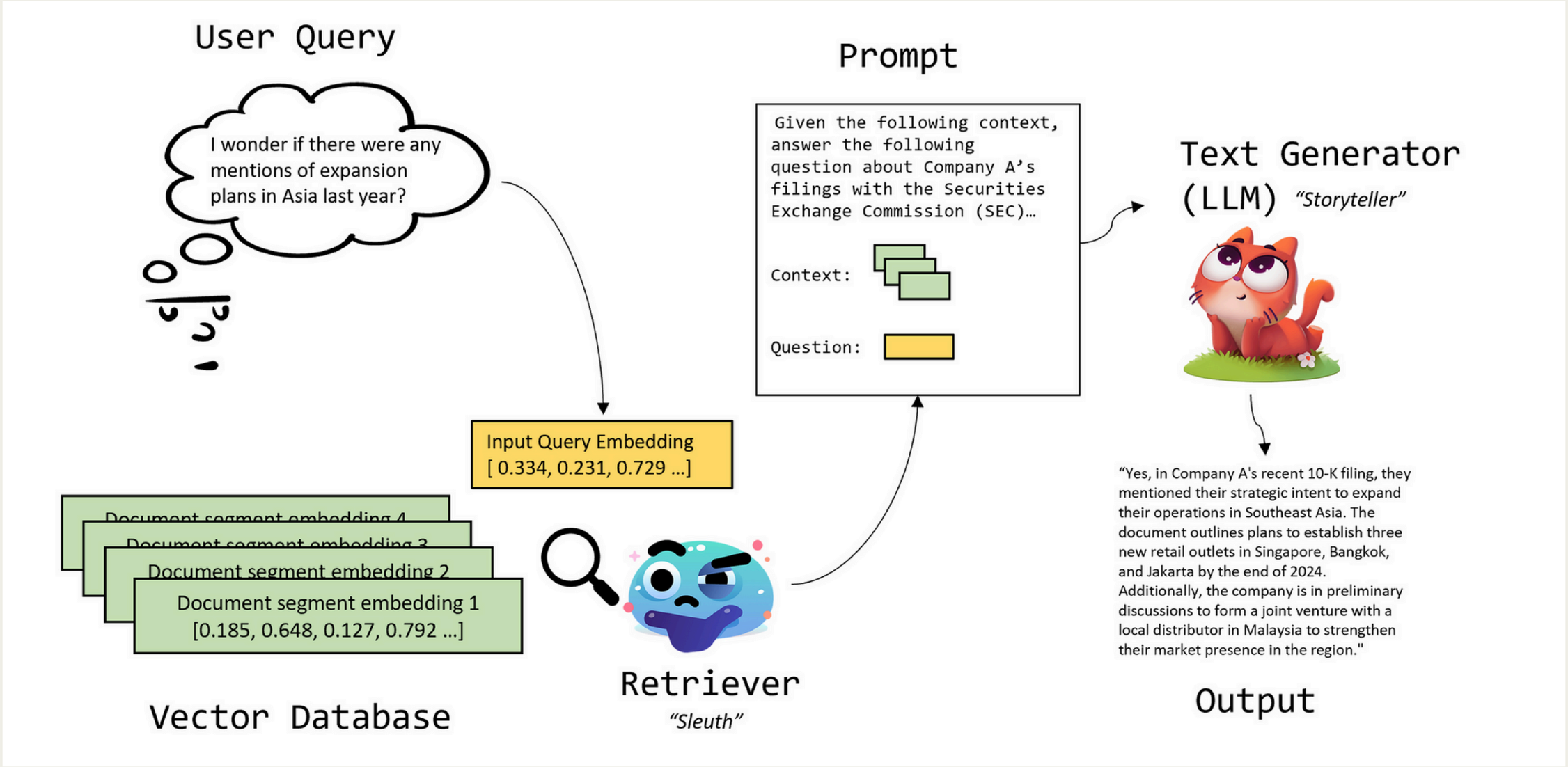
Introduction

1. Problem & Motivation
    - Many Vietnamese search for medical information online but face difficulties.
    - Information is often inaccurate, hard to understand, or hallucinated by AI models.
2. Research Objectives
    - Build a reliable and easy-to-understand Vietnamese medical Q&A system using RAG.



Theoretical Foundation

RAG Overview



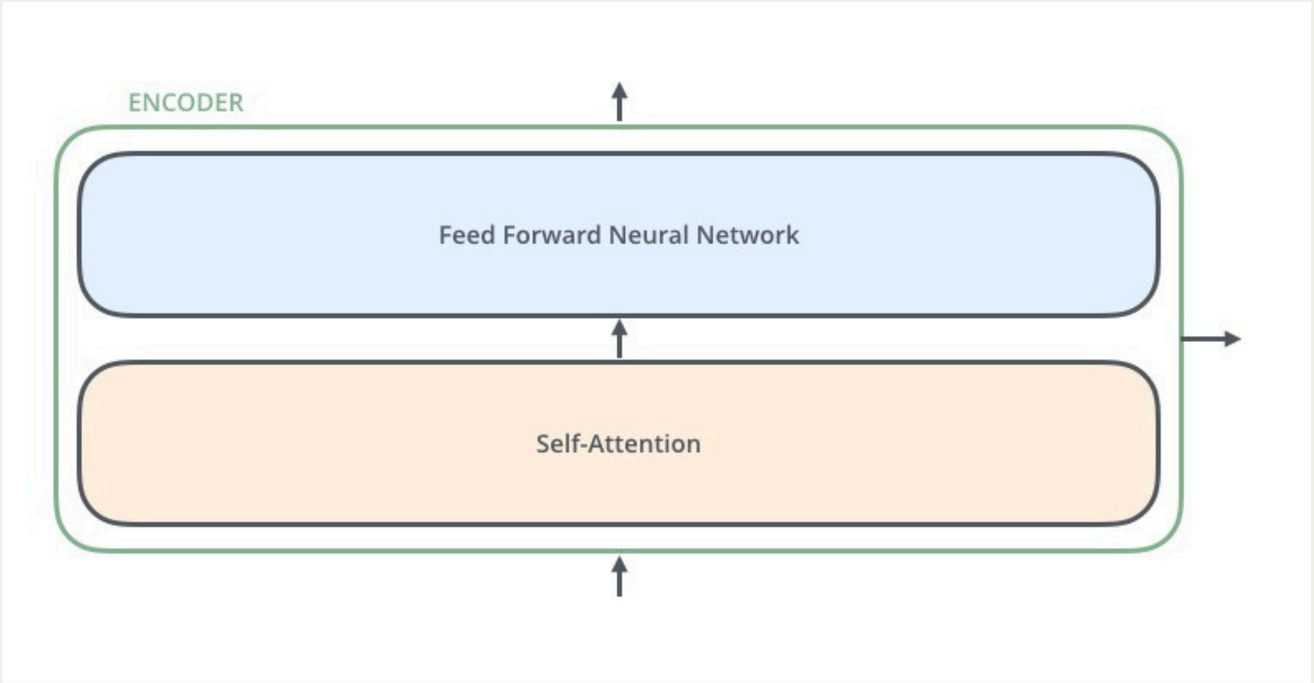
Retrieval Methods



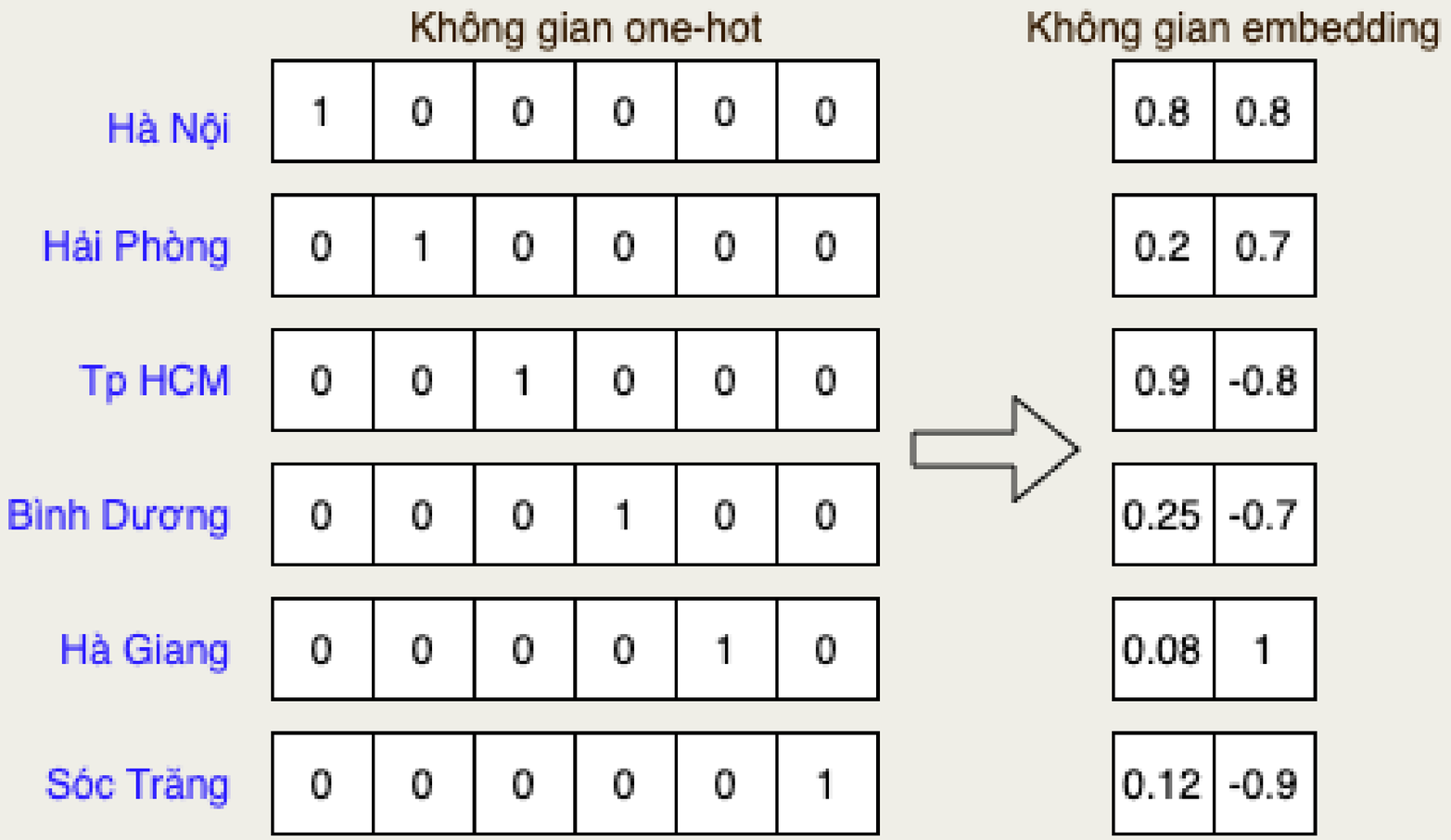
- Embedding Model
- BM25
- Knowledge Graph

Embedding Model

1. Text Embedding
2. Semantic Search
3. Use Cases
4. Architecture

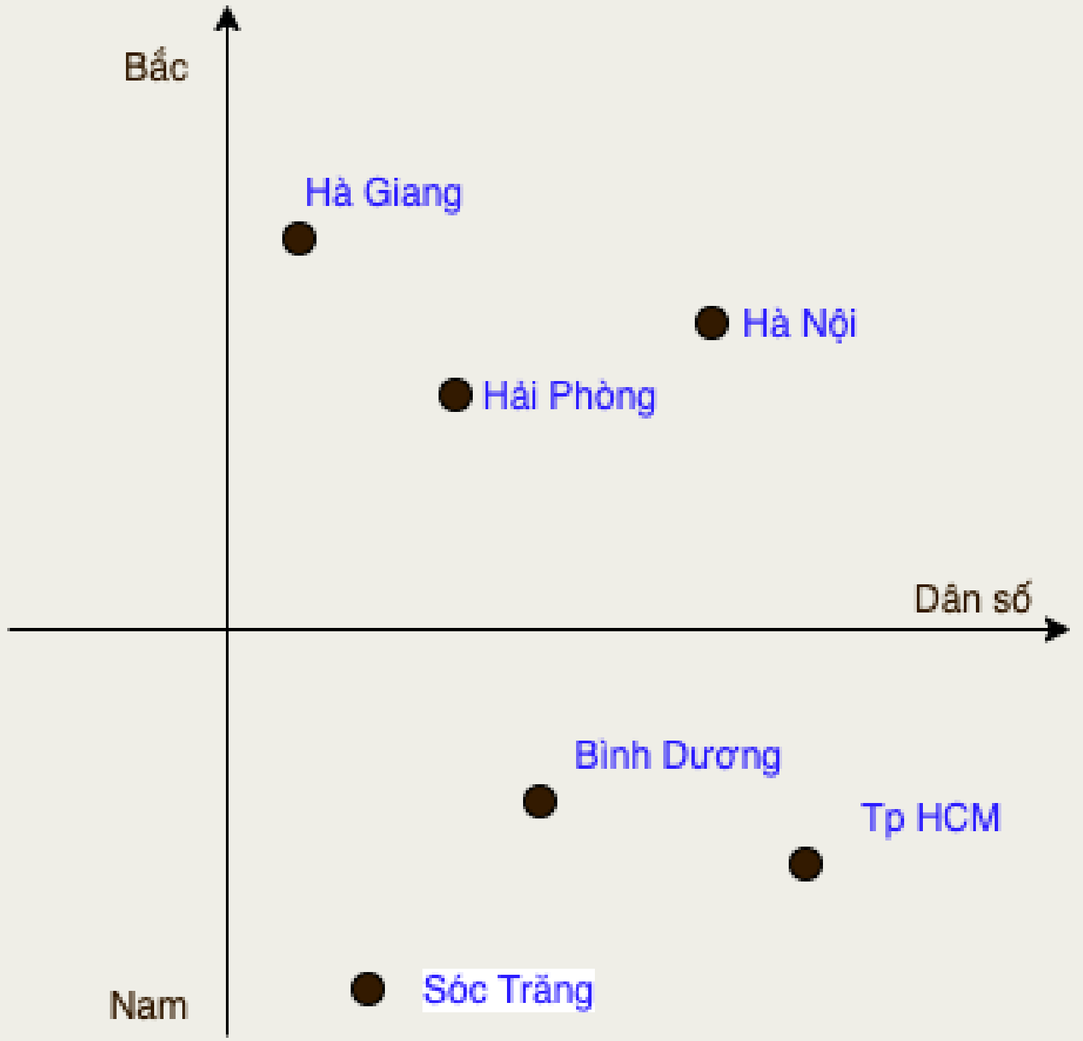


Text Embedding



Semantic Search

1. Distance between two vectors measures their relatedness. Smalldistances suggest high relatedness.
2. Large distances suggest low relatedness.



TEXT EMBEDDING USE CASES

Embeddings are commonly used for:



Semantic Search where results are ranked by relevance to a query string



Clustering where text strings are grouped by similarity



Recommendations where items with related text strings are recommended



Anomaly detection where outliers with little relatedness are identified



Diversity measurement where similarity distributions are analyzed

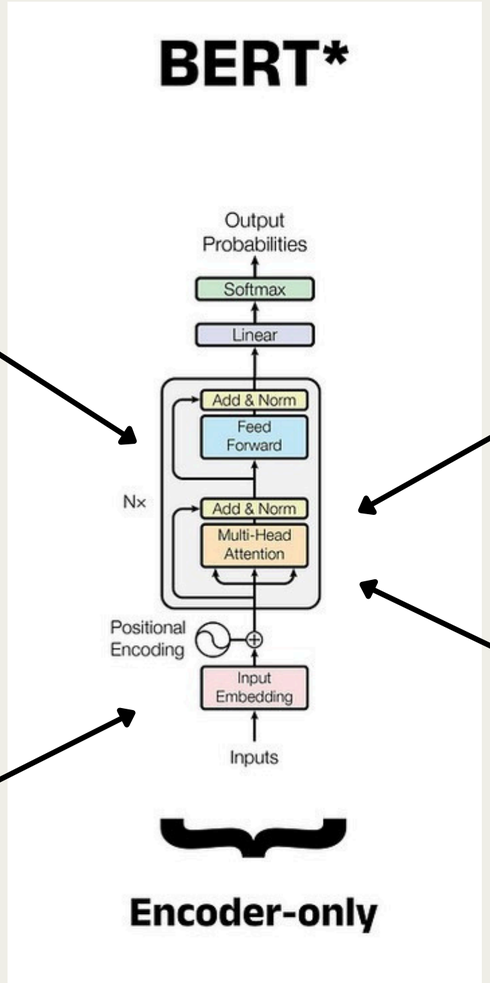


Classification where text strings are classified by their most similar label

Architechture

The residual

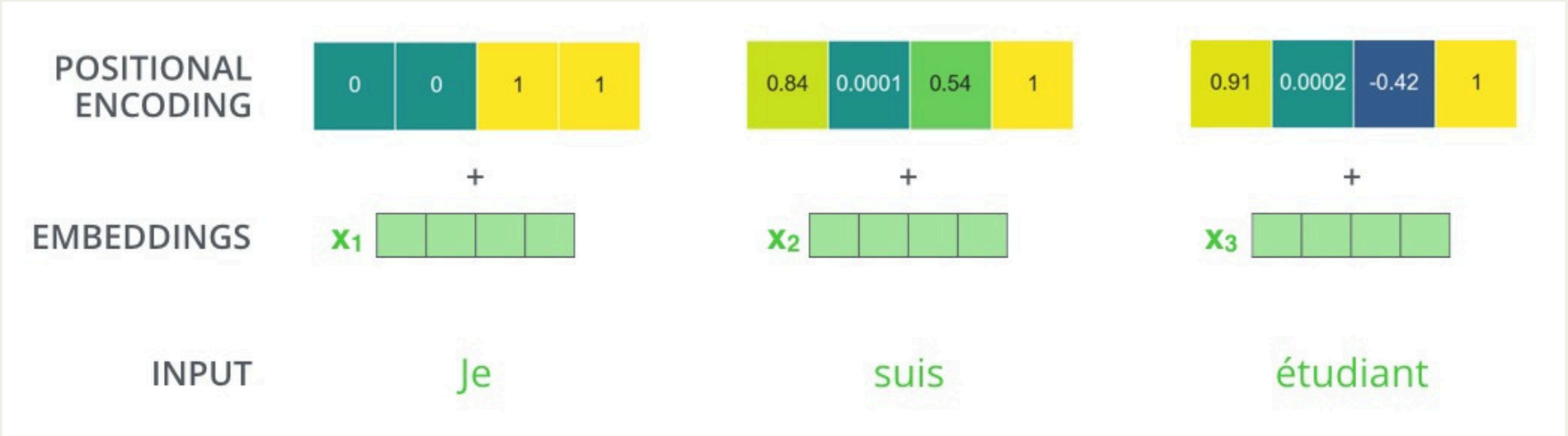
Positional Encoding



Multi-head

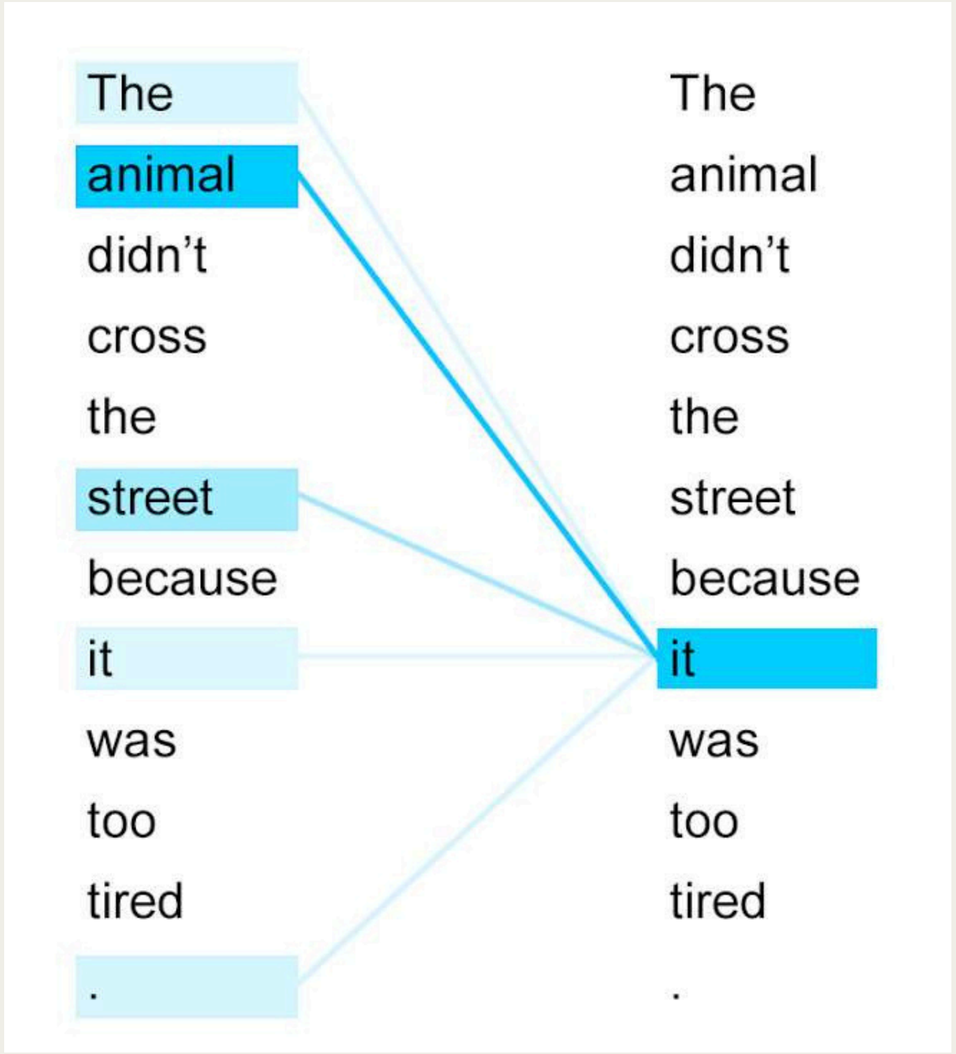
Self Attention

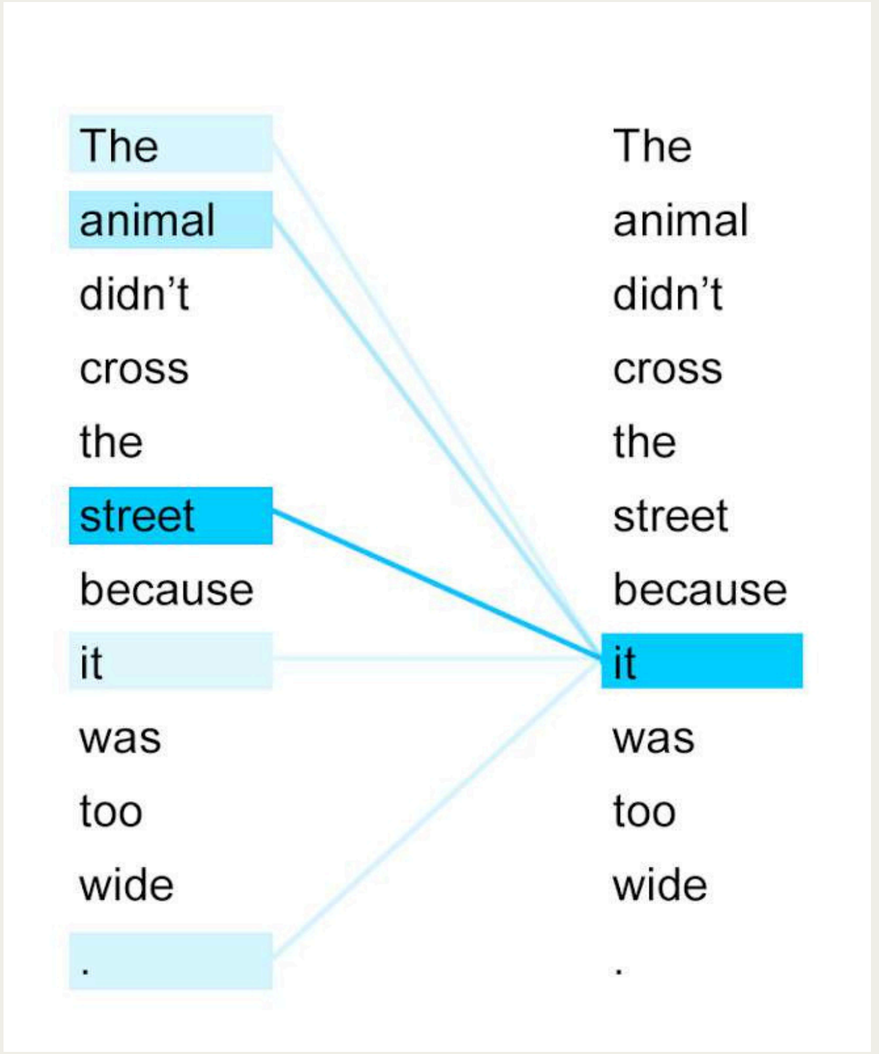
Positional Encoding



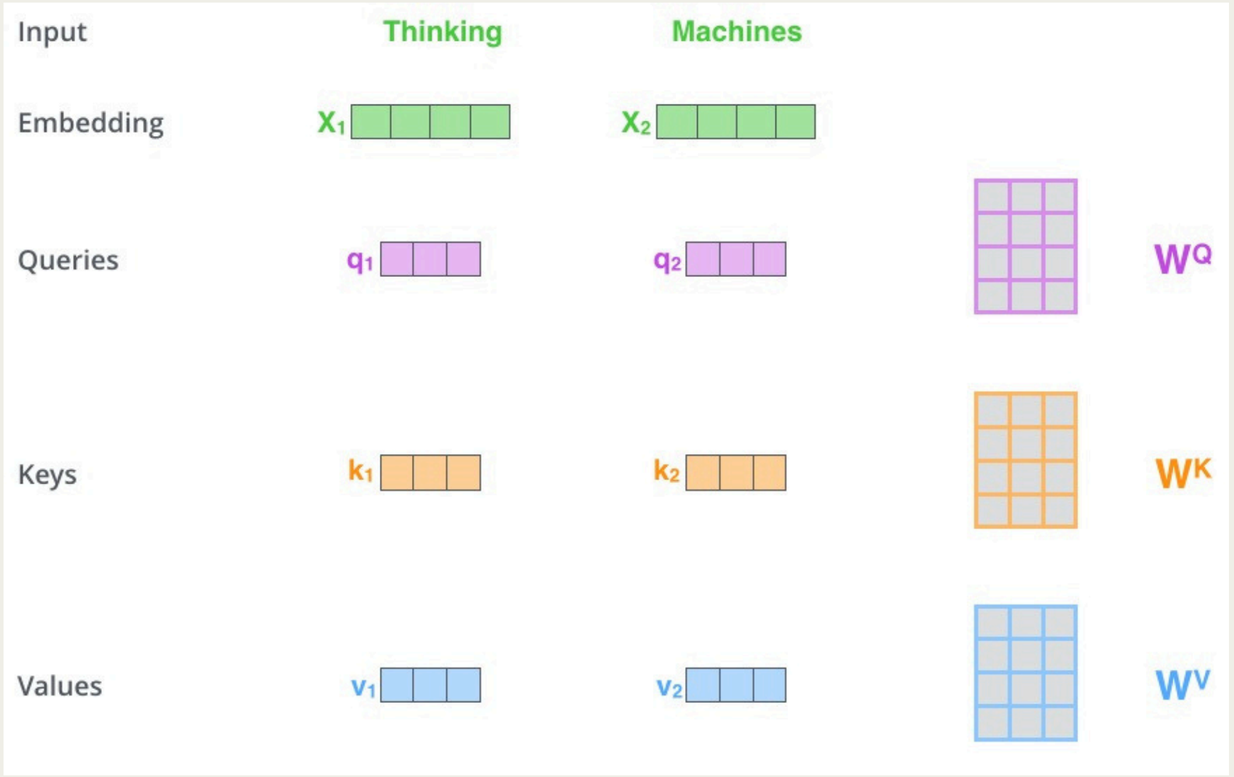
$$p_{i,j} = \begin{cases} \sin\left(\frac{i}{10000^{\frac{j}{d_{emb_dim}}}}\right) & \text{if } j \text{ is even} \\ \cos\left(\frac{i}{10000^{\frac{j-1}{d_{emb_dim}}}}\right) & \text{if } j \text{ is odd} \end{cases}$$

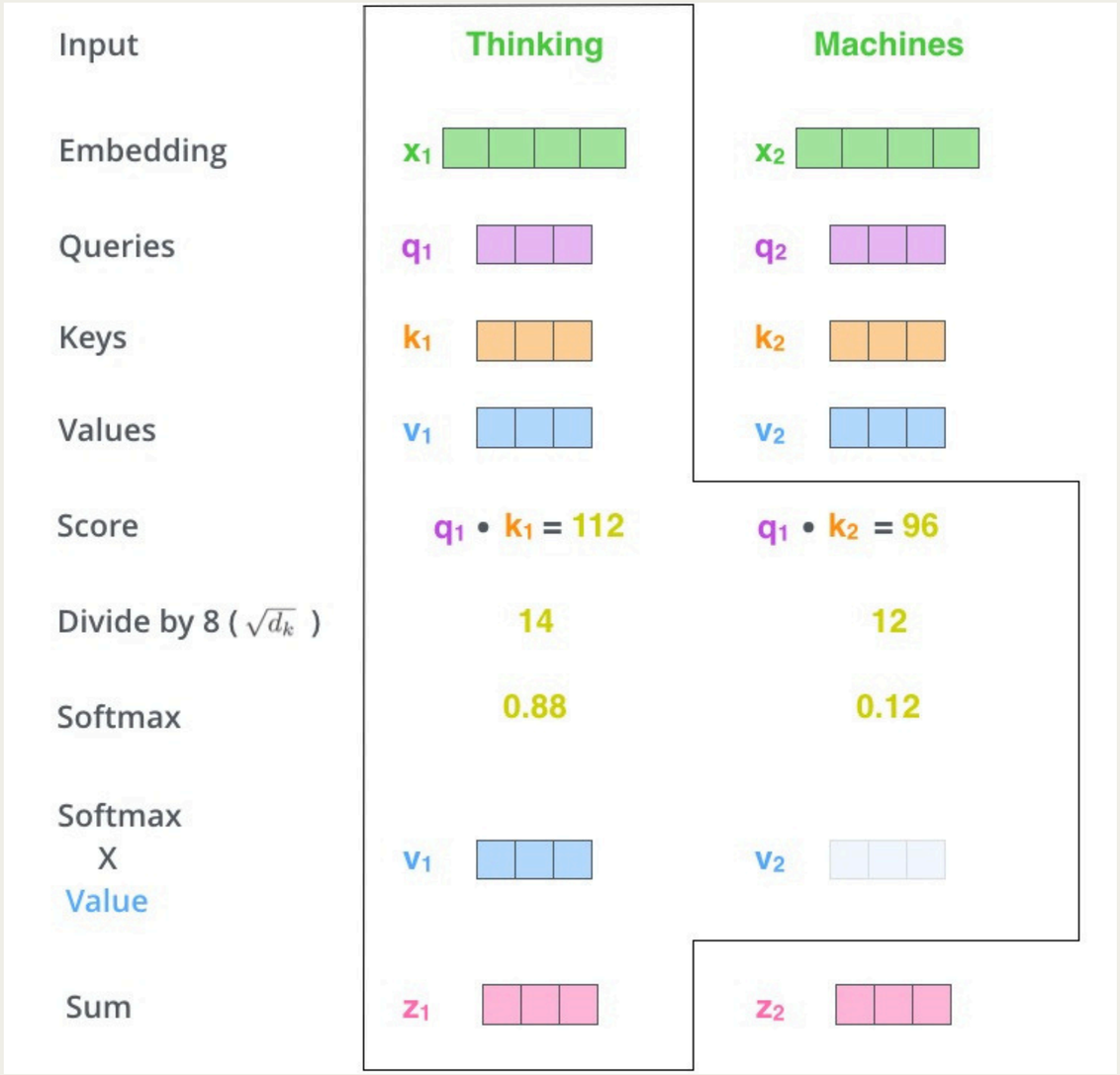
Self Attention





Self Attention





Multi-head

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X

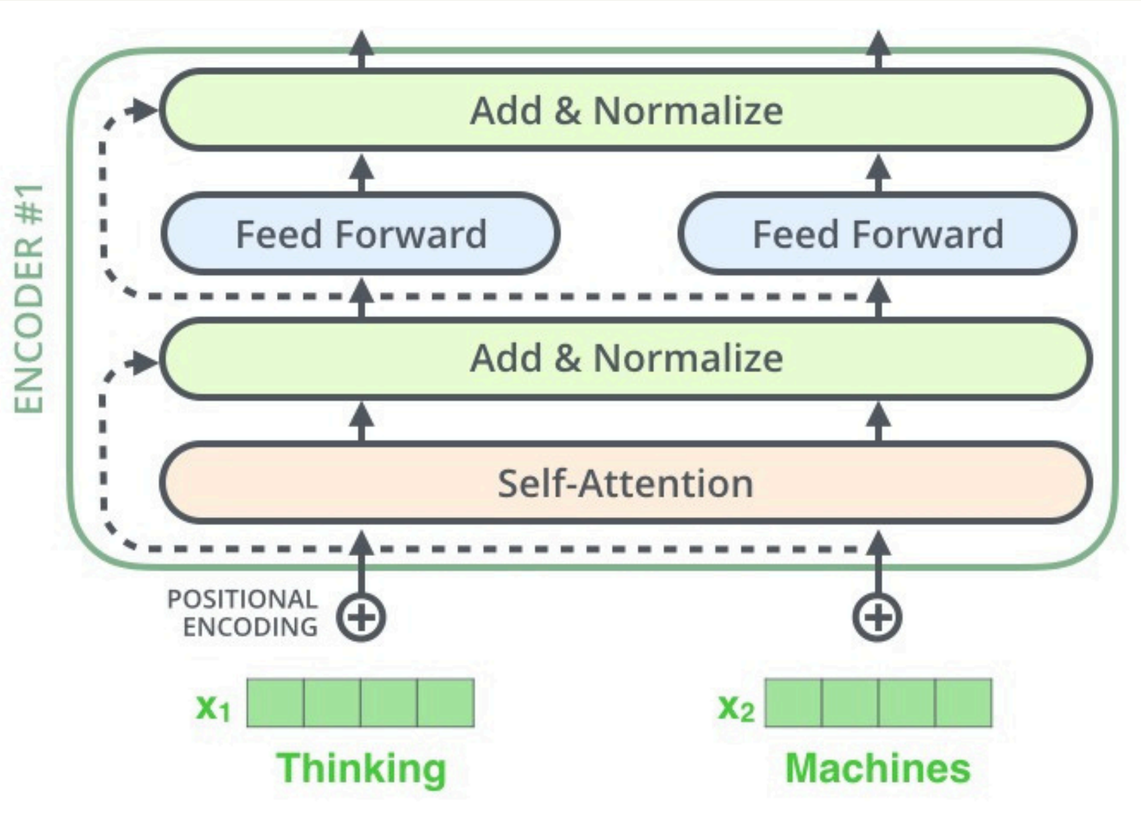


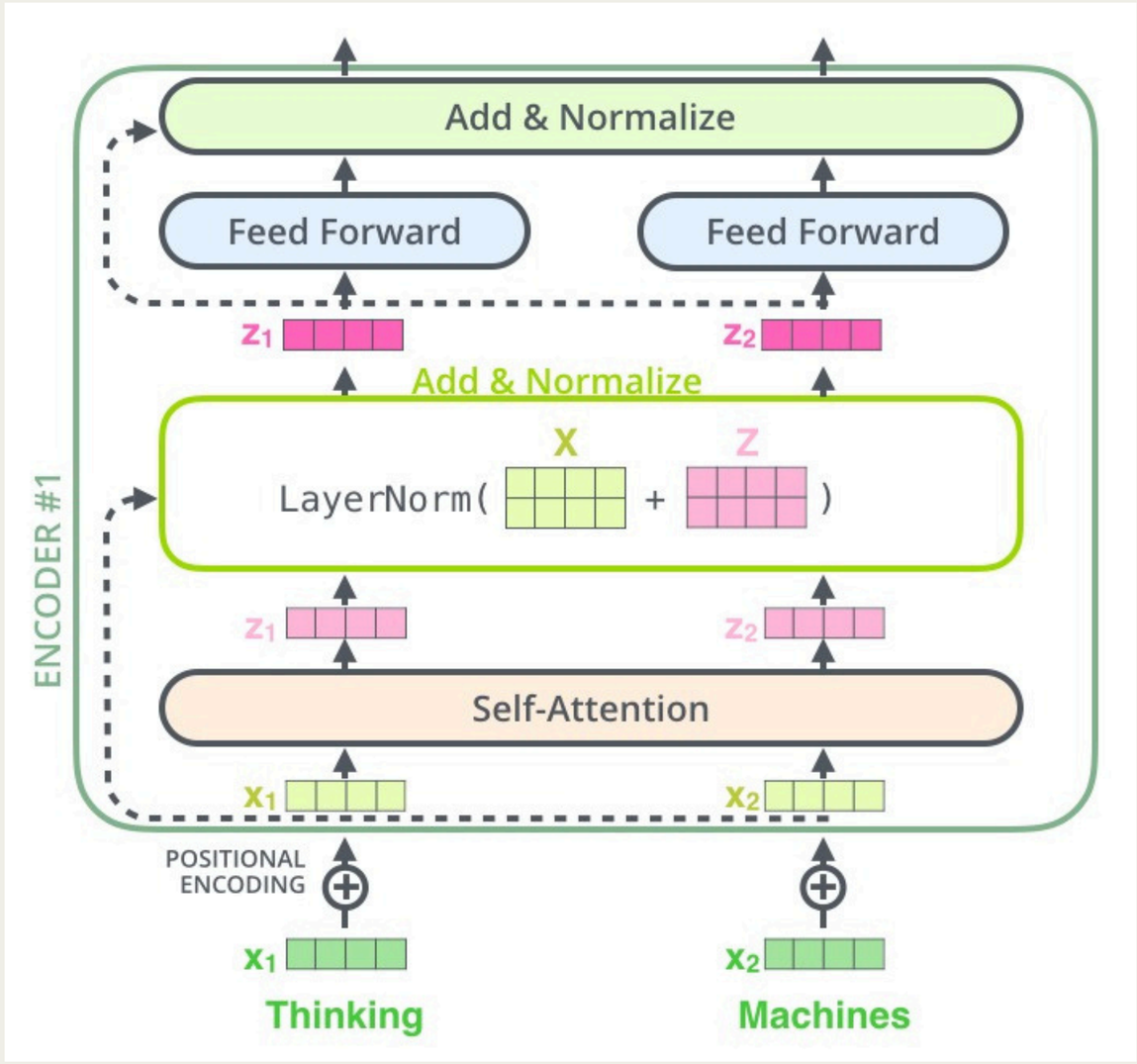
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



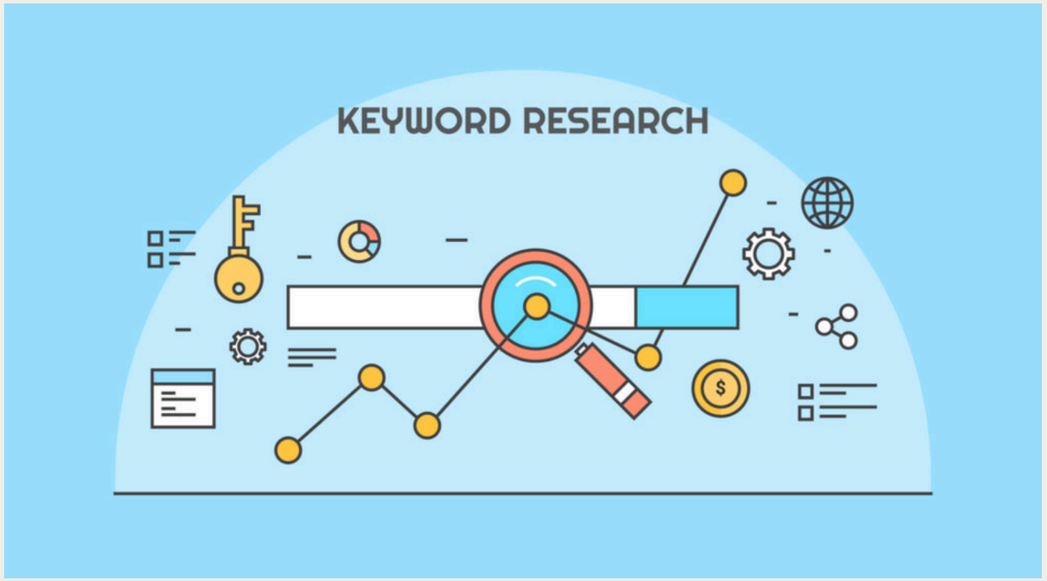
The residual





Keyword Search - BM25

1. Keyword Search = lexical matching using an inverted index.
2. Strengths: high precision for medical terms (disease, drug, symptom).
3. Weaknesses: brittle to synonyms/paraphrase; needs good preprocessing.
4. In HealthRAG, we use BM25 as our keyword search method,



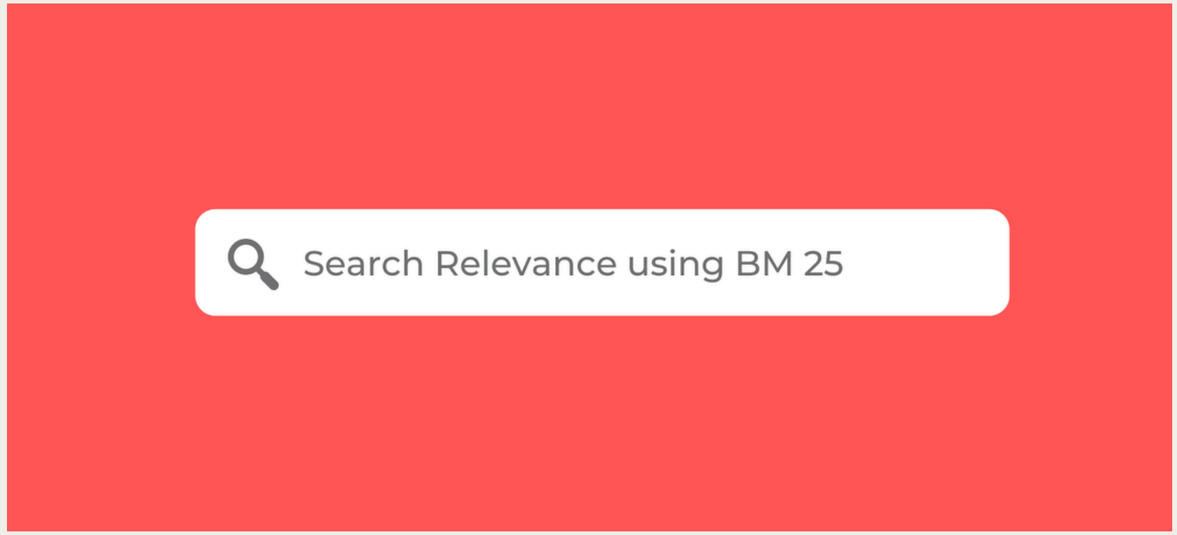
Keyword Search - BM25

1. Probabilistic ranking function improving over TF-IDF.
2. Two key ideas: TF saturation & length normalization.
3. Tunable parameters: k1 (TF saturation), b (length norm).
4. Cited in thesis as algorithm [12] (Robertson & Zaragoza 2009

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Keyword Search - BM25

1. Sparse retrieval: BM25 on an inverted index over the medical corpus.
2. Runs in parallel with dense embeddings & the Knowledge Graph.
3. Outputs are fused by Reciprocal Rank Fusion (RRF).



Keyword Search - BM25

BM25 Implementation

Step 1: Preprocessing

- Load ~5,800 QA pairs
- Corpus = all answers
- Tokenization (.split() in this project)

Step 2: Indexing

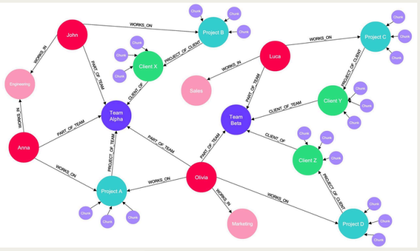
- BM25 builds inverted index of tokens
- Stores term frequencies and document lengths

Step 3: Querying & Ranking

- Input question → tokenize
- BM25 scores each document
- Sort docs by score (highest first)

After scoring, take Top-K documents

- Example:
  - K=1 → best guess (most precise)
  - K=3, 5, 10 → broader coverage (higher recall)
- Evaluate system at different K values to measure trade-off between Precision and Recall



Knowledge Graph

Motivation & Problem

Heterogeneous and fragmented healthcare data

Complex and multi-level relations among diseases, symptoms, drugs, and treatments

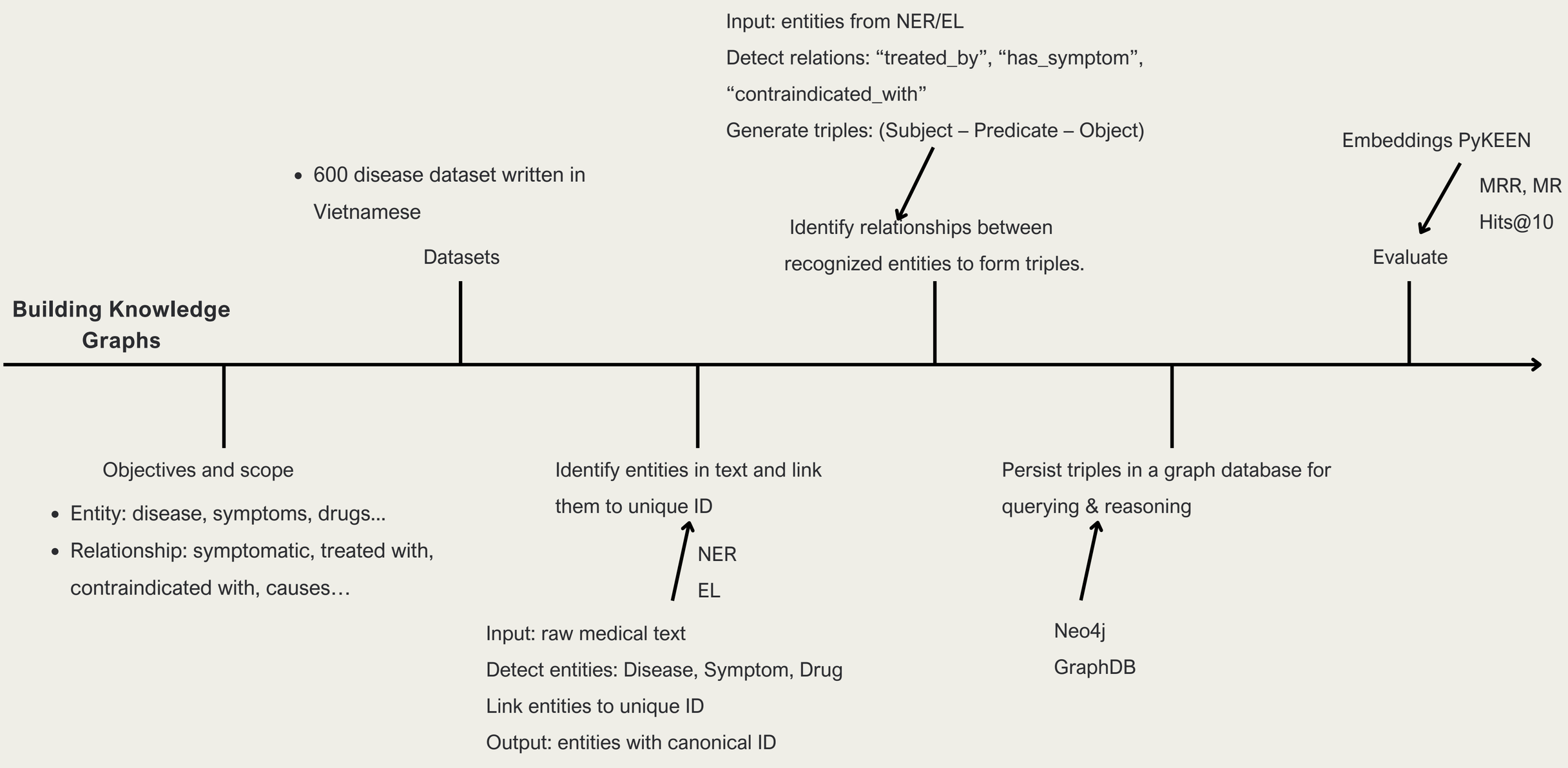
Enable semantic search and reasoning over complex relationships

Reduce inconsistencies and errors by unifying medical terminology and logic constraints

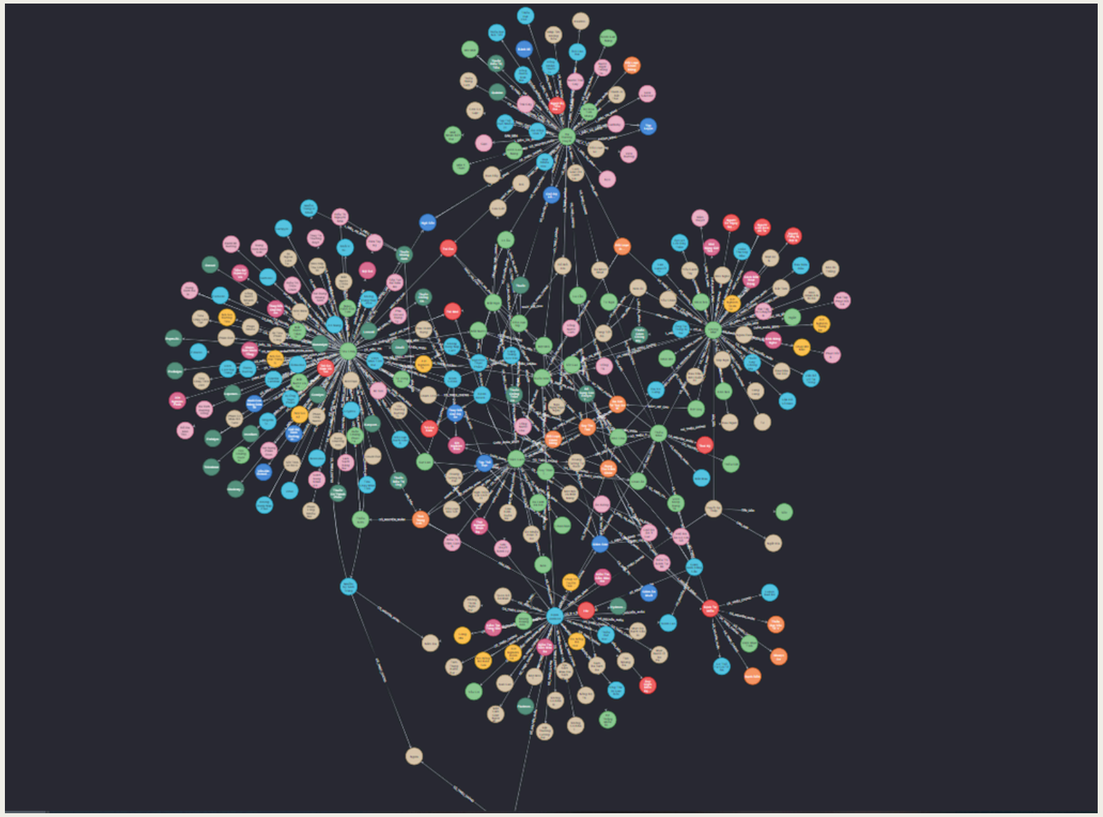
Enhance accuracy and explainability in AI/RAG systems

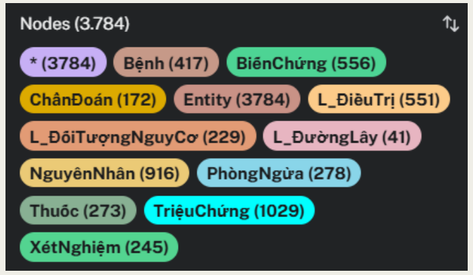
Scalable and updatable structure for evolving medical knowledge

Knowledge Graph

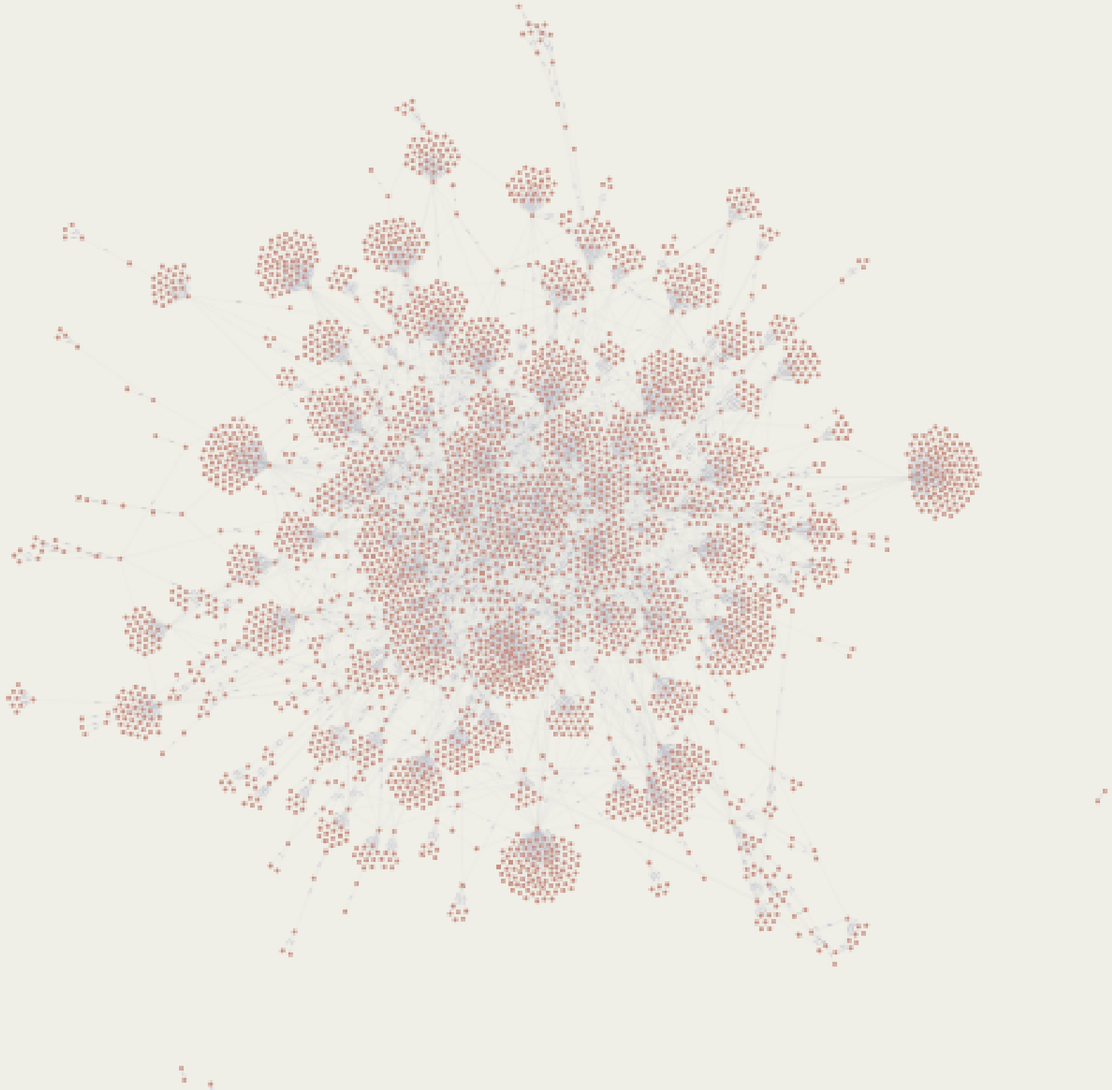


Knowledge Graph



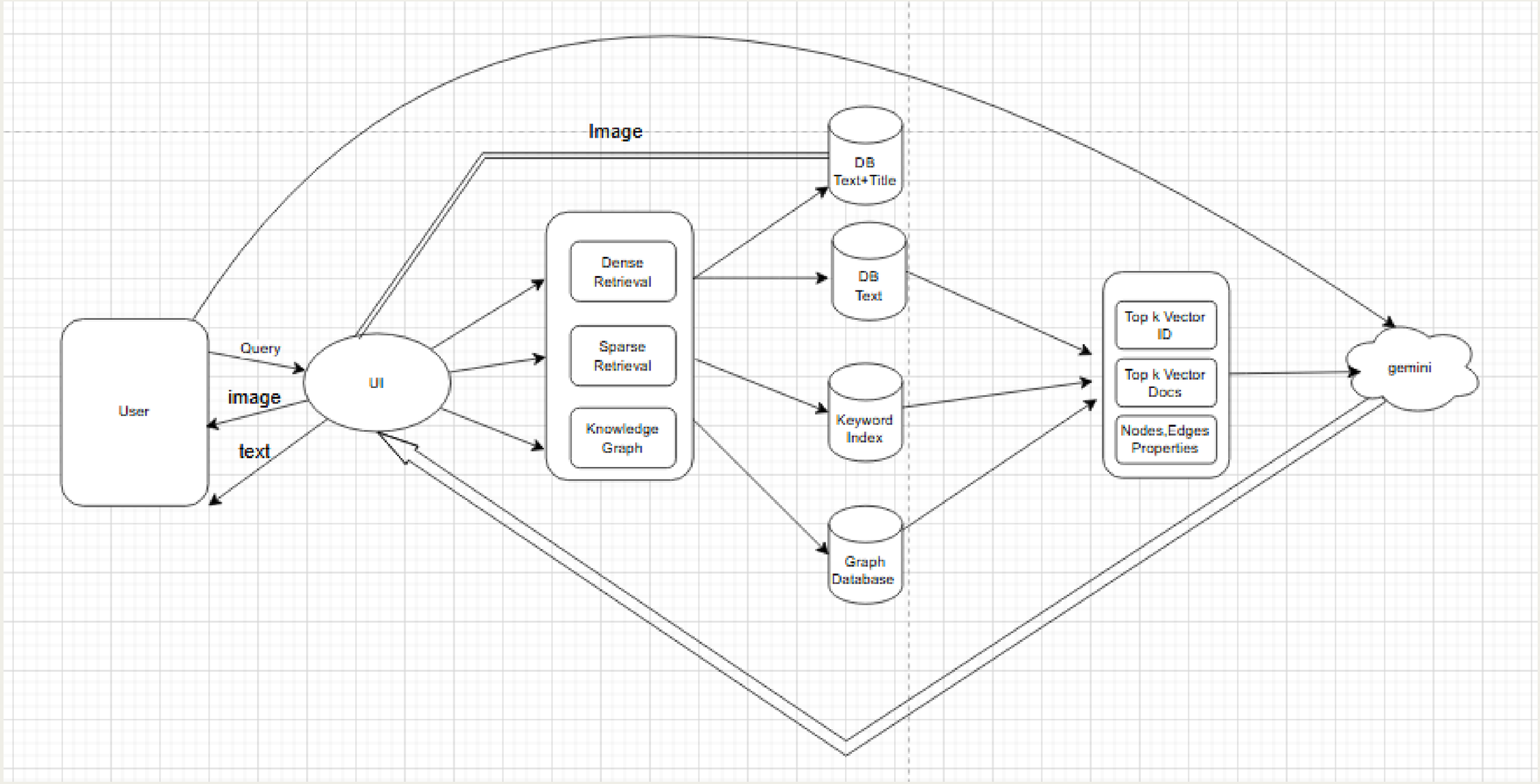


- Nodes 22.704
- Relationships 22.789



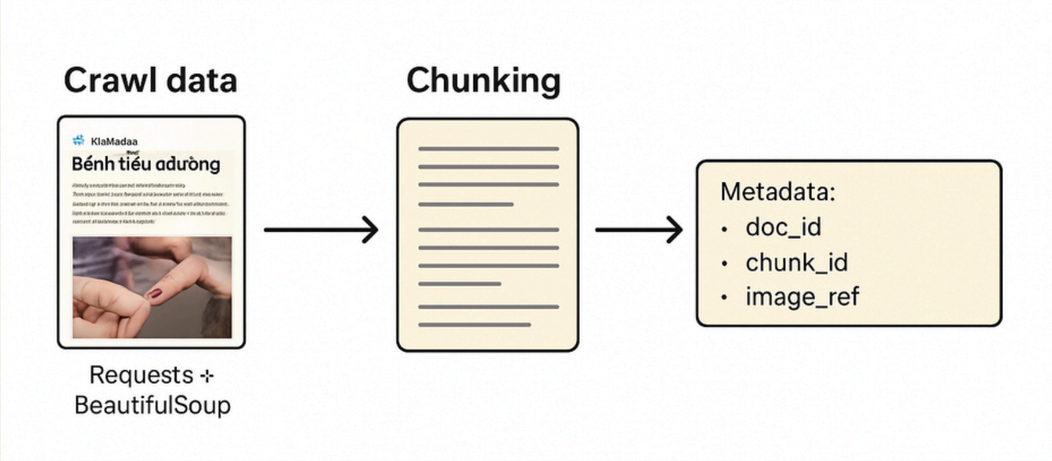
Methodology

System Architecture



Data Collection & Preprocessing

1. Crawl Data
   - Request HTML: use requests.get(url)
   - Parse DOM: with BeautifulSoup
   - Save
2. Chunking
   - Split text into smaller chunks by Paragraph
3. Create QA Pairs for Testing
   - LLM-assisted QA + Human Cheking



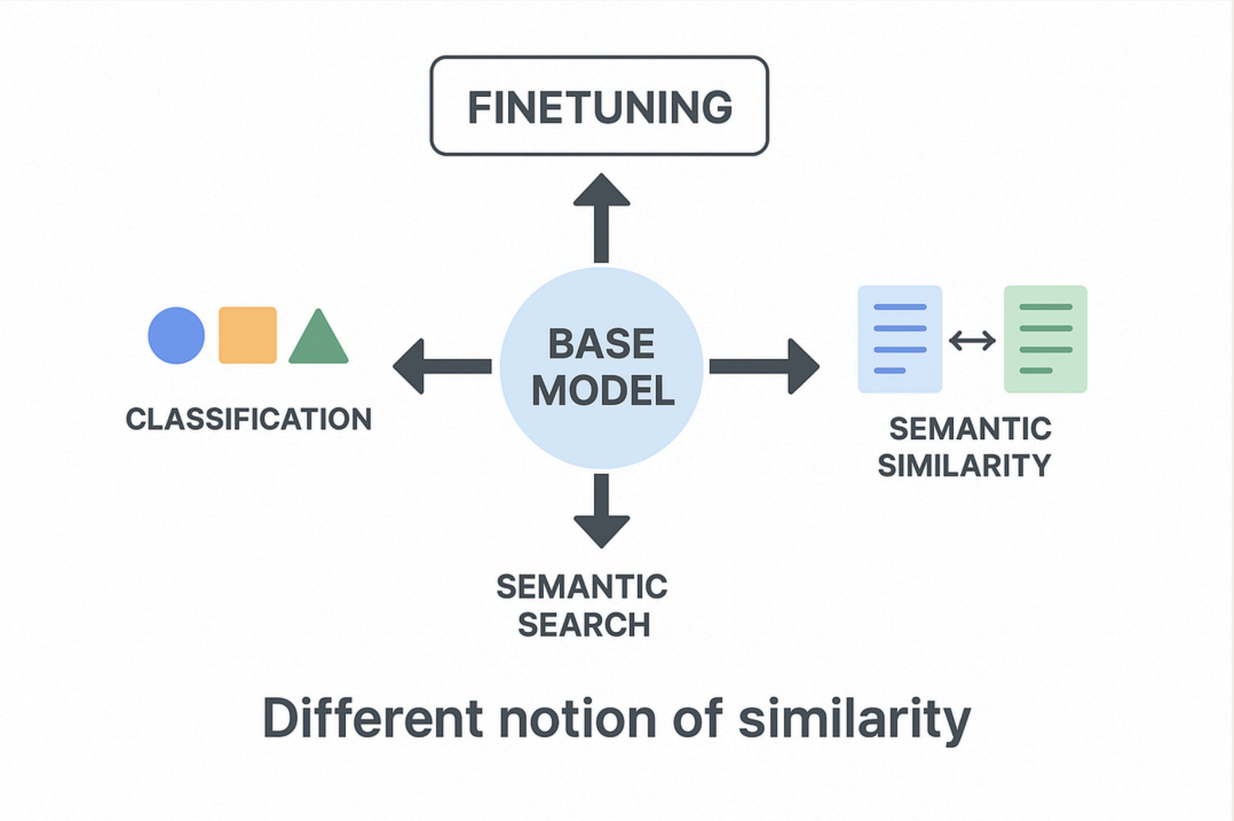
Model Fine-tuning

1. Why Fine-tuning
2. Dataset Format
3. Loss Functions
4. Training Arguments
5. Metrics



Why Fine-Tuning

1. Different notions of similarity
    - Classification
    - Semantic Similarity
    - Semantic Search
2. Different training data
    - Base models are trained on general data



Dataset

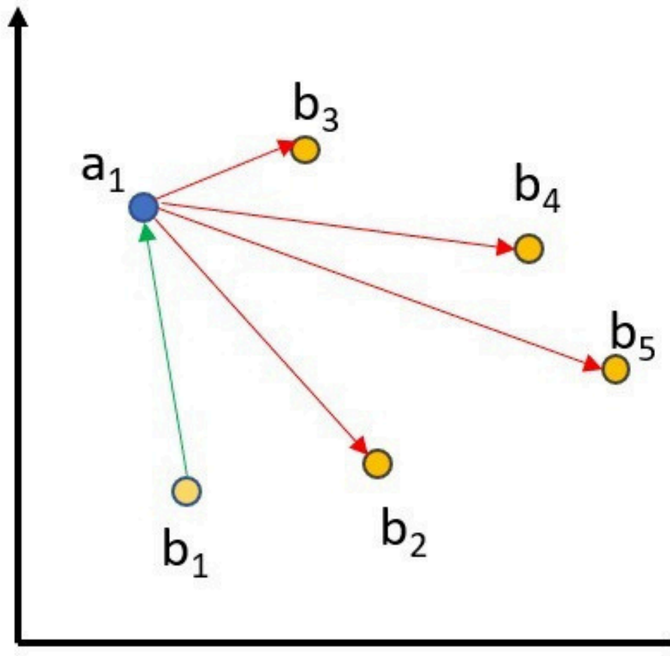
1. Datasource
    - Youmed.com
    - Contain 610 articles
2. Format
    - Question - Answer

```
{"question": "<question>, "context": <"relevant context to answer">
{"question": "<question>, "context": <"relevant context to answer">
{"question": "<question>, "query ↔ document, nkong so document with document.)
```

MultipleNegativesRankingLoss

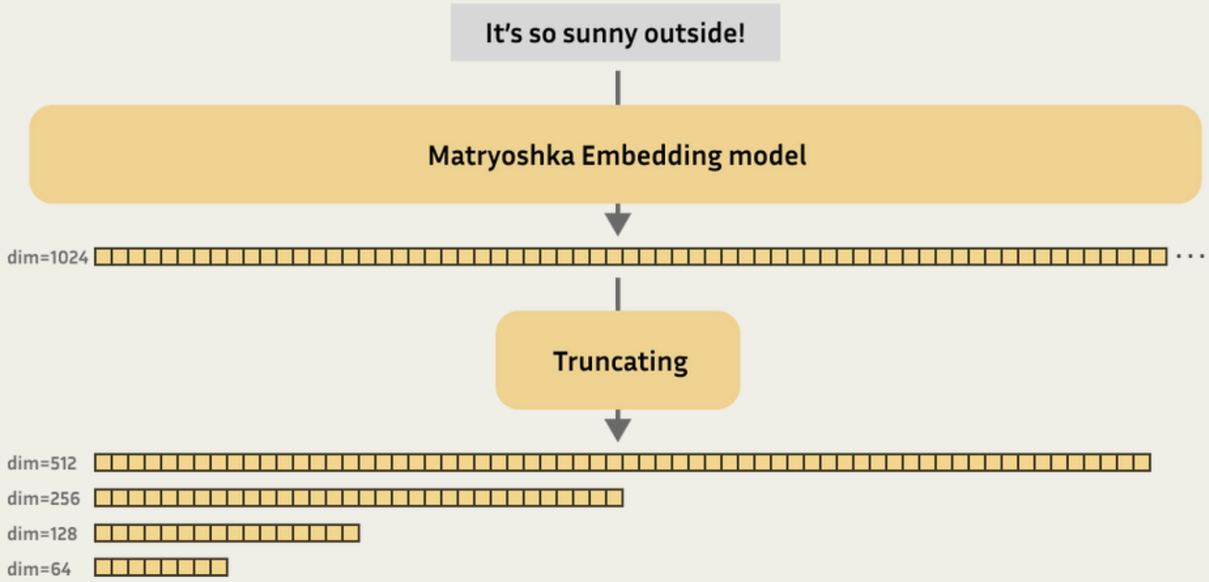
1. Pairs
   - (a1,b1),…,(an,bn)
2. Similar vs. Dissimilar
   - (a_i, b_i) → similar sentences
   - (a_i, b_j) → dissimilar sentences
3. Distance Adjustment
   - Distance between (a_1, b_1) → reduced
   - Distance between (a_1, $b_{2\ldots5}$) → increased

$$\mathcal{L} = -\log\left(\frac{\exp(\text{sim}(a,p)/\tau)}{\sum_{i=1}^{N}\exp(\text{sim}(a,n_i)/\tau)}\right)$$



Matryoshka loss

1. Purpose
   - Evaluates full-size embeddings
   - Checks quality at smaller dimensions
   - Sizes: 768, 512, 256, 128, 64
2. How it works
   - Compute loss for each dimensionality
   - Sum them into a final loss
   - Optimizer updates weights



Training Arguments

1. General Setup
    - output_dir
    - num_train_epochs
    - per_device_train_batch_size
    - gradient_accumulation_steps
    - per_device_eval_batch_size

2. Optimization
    - learning_rate
    - lr_scheduler_type
    - optim
    - warmup_ratio
    - bf16
    - batch_sampler

3. Evaluation & Logging
    - output_dir
    - num_train_epochs
    - per_device_train_batch_size
    - gradient_accumulation_steps
    - per_device_eval_batch_size

Metrics

$$\text{MRR@k} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank}_i}$$

$$Hits@k = \frac{1}{N} \sum_{i=1}^{N} \text{hit}_i@k$$

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}, \quad \text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

$$\text{Precision@k} = \frac{\text{Number of relevant documents in top k}}{\text{k}}$$

$$\text{Recall@k} = \frac{\text{Number of relevant documents in top k}}{\text{Total number of relevant documents}}$$

$$\text{Accuracy@k} = \frac{\text{Number of queries with correct document in top k}}{\text{Total number of queries}}$$

Results & Analysis

Before and After finetune (Best dim 128)

Metric	Before Fine-Tuning	After Fine-Tuning	Improvement (%)
Accuracy@1	0.5489	0.7781	22.92
Accuracy@3	0.7994	0.9935	19.40
Accuracy@5	0.8483	0.9993	15.10
Accuracy@10	0.8920	1.0000	10.80
Precision@1	0.5489	0.7781	22.92
Precision@3	0.2665	0.3312	6.47
Precision@5	0.1697	0.1999	3.02
Precision@10	0.0892	0.1000	1.08
Recall@1	0.5489	0.7781	22.92
Recall@3	0.7994	0.9935	19.40
Recall@5	0.8483	0.9993	15.10
Recall@10	0.8920	1.0000	10.80
NDCG@10	0.7350	0.9157	18.07
MRR@10	0.6831	0.8860	20.29
MAP@100	0.6866	0.8860	19.94

Keyword Search - BM25

1. Solid performance: Our BM25 system effectively finds relevant documents, with a strong ability to place them in the top ranks.
2. Improved coverage: The system's performance improves significantly as the search scope (k) increases, showing robust retrieval.
3. High-quality foundation: Its high MRR score proves it's a reliable base for an effective information retrieval system.

Metric	@1	@3	@5	@10	MRR
Precision	0.4769	0.2361	0.1546	0.0828	-
Recall	0.4769	0.7082	0.7731	0.8278	-
NDCG	0.4769	0.6158	0.6426	0.6606	-
Accuracy	0.4769	0.7082	0.7731	0.8278	-
MRR	-	-	-	-	0.6114

Knowledge Graph

Evaluate

Check the structural quality of the KG and its embeddings.

Assess the KG's ability to predict missing triples.

Reduce costs by avoiding calls to large LLMs.

Evaluation Method: Link Prediction via PyKEEN:

Train embedding models: RotatE

Metrics

MRR (Mean Reciprocal Rank)

Hits@K (K = 1, 3, 10)

Knowledge Graph

Result

Split triples: train (~80%), validation (~10%), test (~10%)

Hide test triples: model does not see them

Prediction & scoring: model predicts missing triples; compute MRR and Hits@K

Metric	MRR	Hits@1	Hits@3	Hits@10
Value	0.229	0.214	0.227	0.254

Issue: Large KG with many weakly connected or isolated nodes → sparse graph → embedding struggles → low MRR/Hits@K

Solutions:

- Prune low-degree nodes (focus on core entities)
- Add synthetic triples to increase connectivity
- Cluster nodes by type/ontology or use hierarchical KG
- Combine embedding with rule-based methods

Demo Web

Multi-RAG Medical Chatbot

cách chữa trị bệnh zona hiệu quả

Dựa trên thông tin trong hệ thống, không có cách chữa khỏi hẳn bệnh zona, nhưng điều trị kịp thời có thể tăng tốc độ lành vết thương và giảm nguy cơ biến chứng.

Các phương pháp điều trị hiệu quả bao gồm:

- **Thuốc kháng virus**: Acyclovir (Zovirax) và Valacyclovir (Valtrex).
- **Các loại thuốc khác có thể được bác sĩ kê đơn**:
  - Miếng dán Capsaicin (Qutenza)
  - Thuốc chống co giật (gabapentin)
  - Thuốc chống trầm cảm ba vòng (amitriptyline)
  - Thuốc gây tê (lidocaine)
  - Thuốc giảm đau (codein)
  - Thuốc tiêm corticosteroid
  - Thuốc gây tê tại chỗ

Ảnh liên quan:



bệnh_zona.jpg



zona_tai.jpg



Thực_phẩm_nên_dùng_khi_mắc_bệnh_zona_thần_kinh.jpg

Thank you!